Practical-6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

A.Theory:

1. How is JSON fetched, parsed, and rendered?

Answer: JSON is fetched from an API using an asynchronous network request (such as JavaScript's fetch()), then parsed into a usable object using a method like response.json(). Once parsed, the extracted data is dynamically injected into the HTML DOM to render and update the web page content.

Source: MDN

2. Which array methods are used for search, filter, sort, and pagination?

Answer: To search and filter array data, JavaScript uses methods like find() to retrieve a single matching element and filter() to create a new array of all matching items based on a test condition. Sorting is handled using sort() which reorders elements based on an optional comparator function. Finally, pagination is achieved by combining slice() with a page formula like (page - 1) * pageSize to extract a precise chunk of data for display.

Source: MDN

3. How are loading and error states handled?

Answer: Loading states are managed using UI indicators like skeleton screens or spinners while toggling boolean flags during async operations. Error states are handled by catching network exceptions, displaying actionable user-facing

error messages, and providing retry mechanisms to restore normal functionality.

Source: w3School

4. How is modularity maintained in JavaScript files?

Answer: Modularity in JavaScript is maintained by breaking code into discrete, reusable files using ES Modules (import/export) or CommonJS (require/module.exports), which scopes variables locally to prevent global scope pollution. High-level architecture is further sustained through bundlers like Vite or Webpack that resolve dependency trees, enforce single-responsibility principles, and encapsulate private internal logic from public APIs.

Source: Geeksforgeeks

B. Implementation/ Output/Result

In this practical, semester fee data was stored by me in an external JSON file (fees.json) and loaded into the fee webpage using the Fetch API. An async function loadFees() was written, and the fetch() call inside it requested the JSON file from the data folder. The response was checked using response.ok, and if the status was not successful, an error was thrown and caught by the try...catch block, which displayed the message "Could not load fee data." in the table instead of leaving it blank. When the response was valid, response.json() was used to parse the JSON into an array of objects, which was stored in the feesData array. Each object contained fields such as term, amount due, amount paid and status. The parsed data was then passed to the renderFeeTable() function, which looped over the array with forEach() and displayed every record as a table row created dynamically in the DOM.

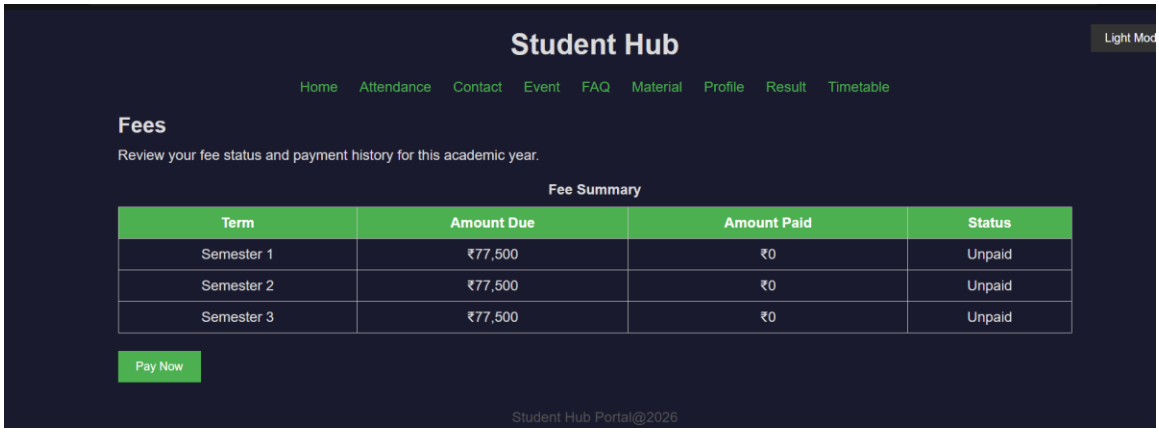
C.Evidence Of Work(Screenshots):

JSON File which has Semester Fee Data:

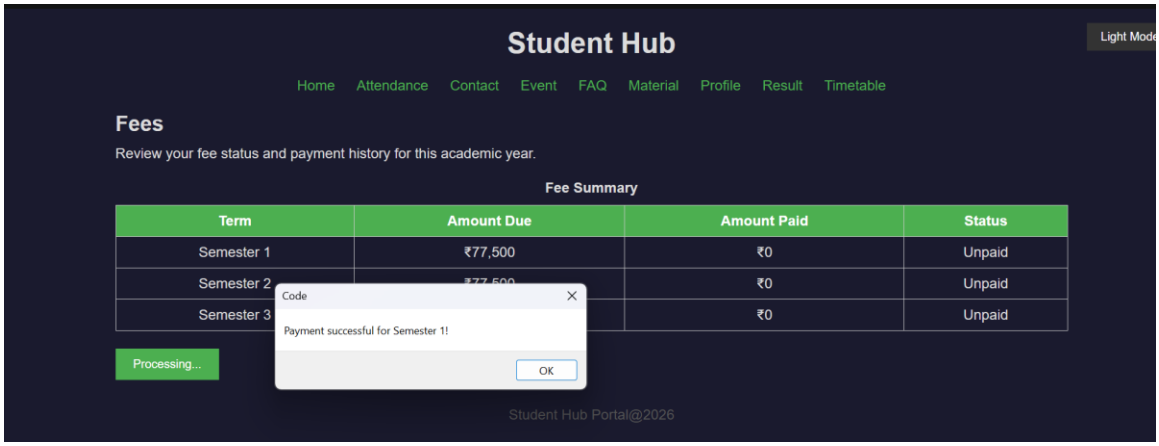
```json
[
  {
    "term": "Semester 1",
    "amountDue": 77500,
    "amountPaid": 0,
    "status": "Unpaid"
  },
  {
    "term": "Semester 2",
    "amountDue": 77500,
    "amountPaid": 0,
    "status": "Unpaid"
  },
  {
    "term": "Semester 3",
    "amountDue": 77500,
    "amountPaid": 0,
    "status": "Unpaid"
  }
]
```

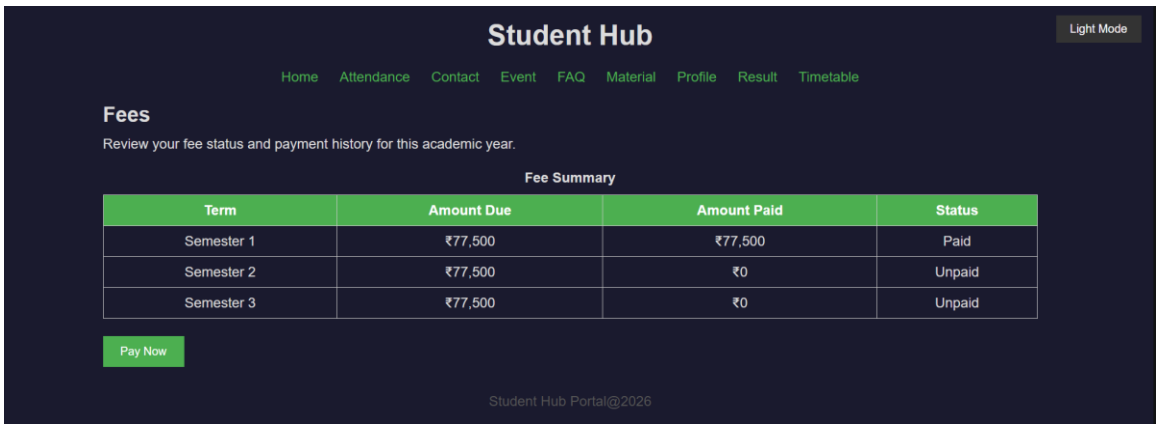
Fee Page:

Initially when loaded:

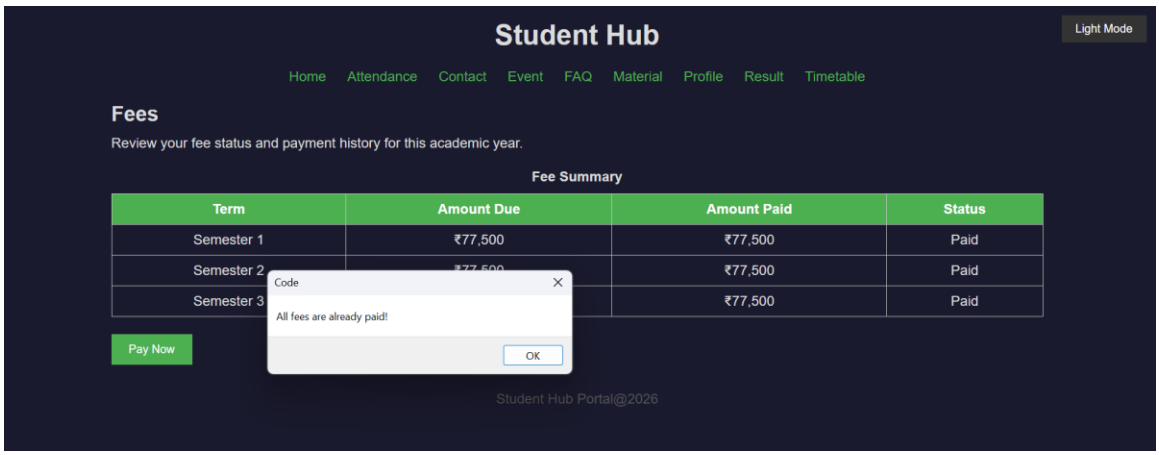


When Clicked Pay Now Button:





When Table is full and clicked Pay now Button:



D. GitHub Repository Link: https://github.com/HimeshMale/Student-Hub-Portal.git

E. Challenge Faced:

A common challenge I faced was that the JSON data did not load at first because the file path was wrong. Another problem was that the table tried to show the data before it had loaded, which I solved by using async/await so the table is rendered only after the JSON is fetched and parsed.

F. Learning Outcomes:

Through this practical, I learned how to fetch data from an external JSON file using the Fetch API and how to parse it with response.json(). I understood how async/await and try...catch help handle asynchronous requests and errors, and why the response.ok check is needed to detect failed requests. I also learned how to render JSON data dynamically in a table using DOM methods and array methods like forEach() and find(). In addition, I learned how to show loading and error messages so that the user is never left with a blank page, and how to keep the code organised by separating fetching, rendering and payment logic into different functions.

Faculty Remarks: